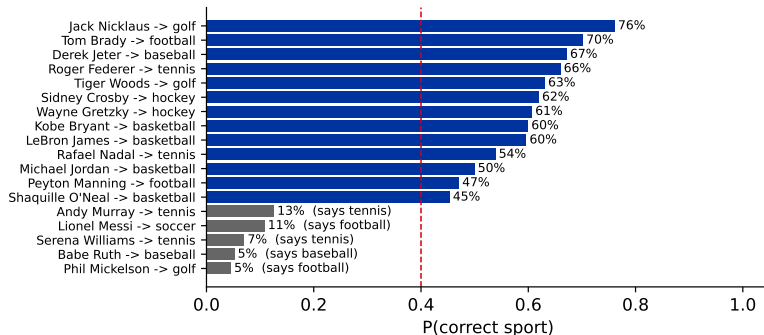


Where Facts Live

Finding a stored fact inside GPT-2 - and rewriting it

The model knows things



"*{name} plays the sport of*" for 18 athletes. GPT-2 small gets **13** right with at least 40% confidence. The rest it half-knows or confuses.

Somewhere in 124 million numbers is "Michael Jordan plays basketball". Today: **where**, and whether we can **change** it.

Predict first

“Michael Jordan plays the sport of” → “basketball”

Three questions. Commit to an answer for each:

1. Is the fact stored in **attention** or in an **MLP**?
2. At which word - on “**Jordan**”, or on the last word “**of**”?
3. In the **early**, **middle** or **late** layers?

Hint from *Does it actually do that?*: pasting one vector from a Tom Brady run into the Jordan run changed the sport only when pasted *at the name* in layers 1-8, or at the last word from layer 10 on.

What we need from the core lectures

- ▶ **The residual stream is a sum** (*Opening the box*): every attention head and every MLP reads the stream and adds its output.
- ▶ **Patching** (*Does it actually do that?*): run one input, paste in an activation from another, and see whether the answer follows the pasted value.

Those two lectures were almost entirely about **attention heads**. Today is about the other half of every block - the **MLP** - and the claim that this is where facts are kept.

Optional lecture. It needs only the two core lectures above.

Outline

An MLP is a lookup table

Causal tracing

Editing a fact

What localisation does not tell you

An MLP is a lookup table

Keys that recognise, values that write

The MLP, written out

At one position, the MLP of a block reads the stream x (768 numbers) and writes back:

$$h = \text{GELU}(x W_{\text{in}} + b_{\text{in}}) \quad (3072 \text{ numbers}), \quad \text{out} = h W_{\text{out}} + b_{\text{out}}$$

Now read $h W_{\text{out}}$ one neuron at a time. W_{out} has one **row** per neuron, so the product is a weighted sum of rows:

$$\begin{aligned} h W_{\text{out}} &= h_1 W_{\text{out}}[1] + h_2 W_{\text{out}}[2] + \cdots + h_{3072} W_{\text{out}}[3072] \\ &= \sum_{n=1}^{3072} \underbrace{h_n}_{\substack{\text{how strongly neuron } n \\ \text{recognised its pattern}}} \cdot \underbrace{W_{\text{out}}[n]}_{\substack{\text{what neuron } n \text{ writes}}} \end{aligned}$$

Neuron n is a **key** (column n of W_{in} : the pattern it fires on) and a **value** (row n of W_{out} : what it adds). An MLP is a soft lookup table with 3072 entries. (Geva et al., 2021)

By hand: three memories

A toy MLP reading a 2-number stream, with ReLU instead of GELU so the arithmetic is easy:

neuron	key (fires on...)	value (writes...)	
1	(1, 0)	"basketball"	a Jordan-like input
2	(0, 1)	"golf"	a Woods-like input
3	(0.5, 0.5)	"is a sport"	any athlete

Input $x = (1, 0.2)$ - mostly Jordan:

$$h = \text{ReLU}(x \cdot \text{keys}) = (1, 0.2, 0.6) \Rightarrow \text{out} = 1 \cdot \text{basketball} + 0.2 \cdot \text{golf} + 0.6 \cdot \text{sport}$$

The output is **mostly** "basketball", a bit of "golf", and a general "sport" push that every athlete gets. A real MLP does this with 3072 neurons at once - and that last column is the catch, as the next slide shows.

Measured: what the MLPs write on “Jordan”

Take the MLP output of each layer at the position of “ Jordan” and read it through the unembedding - which tokens does that single write push up?

layer	rank of “ basketball”	top tokens it pushes
0	1,941	“Jordan”, “Jordan”, “Scion”, “Amon”
2	3,503	“arsity”, “XI”, “chio”, “otto”
4	17,132	“Mania”, “ét”, “Mi”, “TD”
6	498	“opponents”, “enters”, “defenders”, “bats”
8	22,689	“travels”, “lashes”, “suffers”, “scrapped”
10	12,932	“missed”, “misses”, “scored”, “shoots”
11	35,271	“averaged”, “ranks”, “leads”, “joined”

No layer writes “ basketball” itself - its best rank is 498 of 50,257 (layer 6). What the MLPs write on the name is **being an athlete**: opponents, defenders (layer 6); missed, scored, averaged (layers 10-11). The name gets *enriched*; the word “basketball” is produced later, at the end of the sentence.

No single neuron holds the fact

At layer 6, on “Jordan”, rank the 3072 neurons by how much each pushes “basketball”:

neuron	fires	push	its value pushes
N170	+0.85	+1.58	“cornerback”, “Bucs”, “linebacker”, “quarterback”
N2774	+0.76	+1.31	“physique”, “skelet”, “athlet”, “runway”
N437	+2.05	+1.21	(a broken byte token), “ha”, “su”, “Kw”
N2587	+0.97	+1.07	“scorer”, “scoring”, “goalkeeper”, “striker”
N578	+1.67	+0.97	“movement”, “legal”, “ocon”, “organ”

It takes **400 neurons** to supply half of the push towards “basketball”.

And the neurons whose values push “basketball” hardest *anywhere* in the model are about sport in general - one pushes “championships, tennis, tournaments” and fires on Tiger Woods as much as on Jordan. The fact is spread over many neurons, each doing something broader: the superposition story from the features lecture, inside one MLP.

Causal tracing

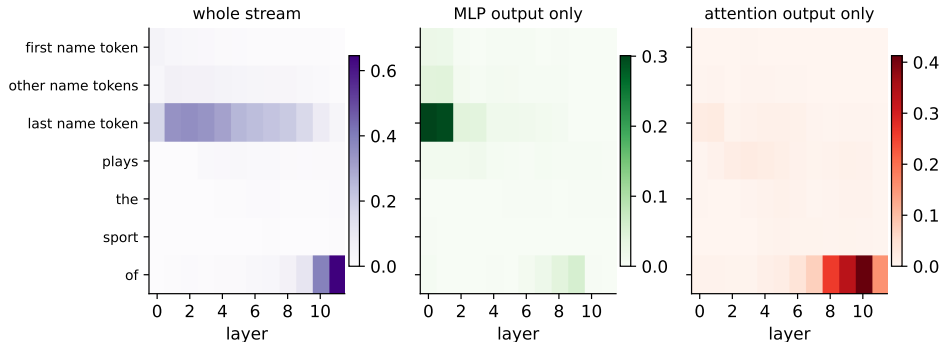
Patching, with noise instead of a second sentence

The recipe

1. **Clean run.** “Michael Jordan plays the sport of” - save every activation.
2. **Corrupted run.** Add random noise to the *name*’s embeddings (twice the typical embedding size - why twice comes later). The model loses the fact: $P(\text{“basketball”})$ falls from **0.60** to **0.028**, averaged over the 13 known facts.
3. **Restore one state.** Run corrupted again, but at one layer and one position copy back the clean value - of the whole stream, or only an MLP’s output, or only attention’s.
4. **Measure** how much of the lost probability comes back.

It is the patching of *Does it actually do that?* with one change: the “other input” is noise, so we do not need a second fact of the same length. (Meng et al., 2022, who named it *causal tracing*.)

The picture



Each cell: share of the lost probability brought back by restoring that one state, averaged over 13 facts (rows: word positions; columns: layers).

Three things stand out. The whole stream at the **last name token** carries up to a third of the fact, from layer 1 to 3. The MLP effect sits in the **very first layers** (0-1), not the middle. Attention matters only at the **last word**, and only late (layers 8-10).

Two sites

Early site: MLPs, on the name

Restoring the MLPs of layers 0-1 at the last name token brings back about **30%** of the lost probability on its own; from layer 2 on, restoring MLPs does almost nothing (at most 4%). In GPT-2 small the lookup happens at the very start - in larger models the same site sits in the middle layers.

Late site: attention, on the last word

Restoring attention at “of” brings back **26%, 33% and 41%** at layers 8, 9 and 10, and almost nothing before layer 7. The attribute is moved to where it is needed only near the end.

The story this suggests: MLPs sitting on the subject **look up** what is known about it, and attention at the end **moves** the attribute the question asks for to where the prediction is made. It is the same shape as the Jordan/Brady patch - now split by component.

Geva et al. (2023) study this “enrich the subject, then extract the attribute” pattern in detail.

The noise is a choice

The original paper adds noise at **3 times** the typical embedding size. On GPT-2 small, the same experiment for Jordan at three noise sizes:

noise	P left	stream at “Jordan” (L1 / L4 / L8)	MLPs L0-2	attention L9-11, “of”
1x	0.202	85% / 81% / 63%	91%	67%
2x	0.041	54% / 59% / 37%	70%	58%
3x	0.010	39% / 23% / 21%	20%	36%

At 3x the noise wrecks the whole sentence, and restoring any one place barely helps - the picture goes blank. At 1x the model still half-knows the answer. We used **2x**: the fact is still almost entirely gone, and restoring can show where it lives. That is a choice, made after looking, and this slide exists so that you know it was made.

Before you trust the picture

- ▶ **The noise is a choice** - the previous slide. Every localisation picture in this field rests on one like it.
- ▶ **Restoring one state is a strong intervention** on a model that has otherwise lost the fact. A large effect says that state is *sufficient* there - not that it is the only place the fact exists.
- ▶ **A small, weak model.** GPT-2 small knows 13 of these 18 facts, and knows them at 45-76%. Large models know far more, and the picture is cleaner there.

It is still the best localisation tool we have for facts. The next section tests whether localisation is good for anything practical.

Editing a fact

If the MLP is a memory, write to it

The plan

Make GPT-2 believe **Michael Jordan plays tennis** - by changing one matrix, W_{out} of the MLP at layer 0 (the layer the tracing pointed at), and nothing else.

1. **The key k^*** : what that MLP's neurons do on “Jordan” - its 3072 hidden activations, averaged over a few different openings of the sentence.
2. **The value change δ** : a 768-number vector that, added to the MLP's output on “Jordan”, makes the model say “tennis”. Found by gradient descent on δ alone - the model's weights are frozen.
3. **The weight change**: alter W_{out} so that k^* now produces δ extra - while every *other* key changes as little as possible.

This is ROME, rank-one model editing (Meng et al., 2022), written from scratch for GPT-2 small in this repo.

Step 3, derived

Find the smallest change ΔW (3072×768) such that the key k^* writes δ more:

$$\min_{\Delta W} \mathbb{E}_k \|k \Delta W\|^2 \quad \text{subject to} \quad k^* \Delta W = \delta$$

where k ranges over keys from ordinary text. With $C = \mathbb{E}[k^\top k]$ (3072×3072 , estimated on 8,192 tokens of Wikipedia text):

$$\mathbb{E} \|k \Delta W\|^2 = \text{tr}(\Delta W^\top C \Delta W)$$

expand the square

$$2 C \Delta W = k^{*\top} \lambda$$

set the Lagrangian's gradient to zero

$$\Delta W = \frac{1}{2} C^{-1} k^{*\top} \lambda, \quad \lambda = \frac{2 \delta}{k^* C^{-1} k^{*\top}}$$

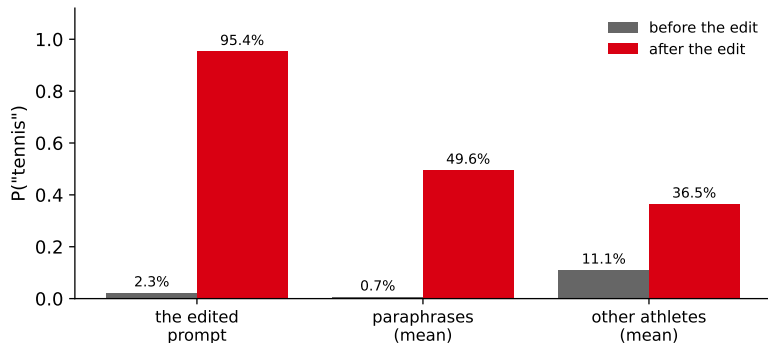
solve, then plug into the constraint

$$\Delta W = \frac{C^{-1} k^{*\top} \delta}{k^* C^{-1} k^{*\top}}$$

an outer product: rank one

Check: $k^* \Delta W = \delta$. C^{-1} steers the change away from directions that ordinary keys often take - that is what protects the other facts.

The edit, graded



$P(\text{" tennis"})$ before and after the edit, on the prompt we edited, on 3 paraphrases of it, and on "{name} plays the sport of" for the 12 other known athletes.

On the edited prompt $P(\text{" tennis"})$ goes from 2% to **95%** and " basketball" from 50% to 0%. All three paraphrases now favour tennis, but by very different amounts (96%, 46%, 7%). And the edit **leaked**: on the 12 other athletes $P(\text{" tennis"})$ rose from 11% to 37%, and only **7 of 12** still get their own sport.

What it says now

Greedy continuation of “Michael Jordan is”, before and after:

before: Michael Jordan is a man who has been a part of the NBA for more than a

after: Michael Jordan is a tennis player who has won the Wimbledon title in 2012 and won

And a related fact the edit never touched:

	before	after
“Michael Jordan played for the Chicago” → “ Bulls”	95%	11%

You can read the Bulls drop either way: a tennis player would not play for the Bulls, so the model is being consistent - or the edit simply damaged what it knew about Jordan. Either way, one rank-one change reached facts it was never told about.

What localisation does not tell you

The tracing pointed at a layer. Is it the right one to edit?

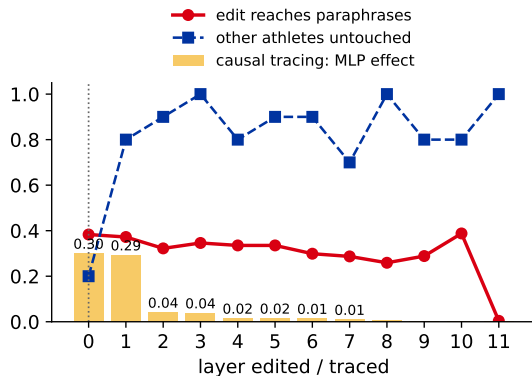
Predict first

Causal tracing said the MLP effect on the name peaks at **layer 0**.

If we run the same edit at **every** layer,
is layer 0 the one where it works best?

“Works best” = the new sport also shows up in the paraphrases, not only on the one sentence we optimised.

The same edit, at every layer



Red: mean P of the new sport on paraphrases. Two edits averaged (Jordan → tennis, Brady → golf).

The layer tracing pointed at, layer 0, is the **worst** place to edit: it reaches the paraphrases as well as any other layer, but wrecks the other athletes - only 20% still right. Layers 1-10 work about as well and do far less damage (70-100% still right); layer 11 does not work at all. Tracing strength and edit quality correlate only weakly over layers (+0.39).

What the literature found

Hase, Bansal, Kim and Ghandeharioun (2023) asked the same question on a much larger model, over thousands of facts: does the layer causal tracing points at predict where editing works?

Their answer: the correlation between the two is **near zero** - for ROME and for several other editing methods. Editing works; the localisation just does not tell you where.

The lesson is bigger than editing. “We found where X is stored” and “we can change X there” are different claims. A picture that localises cleanly is still only an answer to the question it asked - here, “where does restoring help?” - and not to “where should I intervene?”

Recap

- ▶ An MLP is a **key-value memory**: each neuron recognises a pattern (key) and adds a vector (value). On “Jordan” the writes add up to “basketball” - spread over many neurons.
- ▶ **Causal tracing** (patching with noise) puts facts in two places: MLPs on the subject, then attention at the last word.
- ▶ A **rank-one edit** of one MLP matrix can rewrite a fact: Jordan now plays tennis - on the prompt, in paraphrases, and in free text.
- ▶ But the edited world is **inconsistent**, and the layer tracing points at is **not** reliably the best one to edit.

Locating something and controlling it are different skills - and the second one has to be tested on its own.

Next core lecture: features.

We saw today that no single neuron held the fact - the push towards “basketball” was spread over many neurons that each meant something broader. The features lecture asks what the right units are, if neurons are not.